



Actividad práctica en parejas

Objetivo: reemplazar el código que referencia a subclases por código que usa la abstracción, y comprobar en el propio proyecto que eso es lo que permite agregar una clase nueva sin tocar nada.

Los archivos

El código de la actividad está en el repositorio de la cátedra. Si ya está clonado, alcanza con actualizarlo desde la carpeta del repositorio:

```
git pull
```

Si no, se clona una única vez:

```
git clone https://github.com/P00-CR/poo-2026.git
```

También está en archivo comprimido en el classroom. La carpeta de hoy es 07-polimorfismo/actividad.

Todas las clases del catálogo llegan intactas de la clase pasada, con `Producto` abstracta y la interfaz `Descontable` ya escritas. Ninguna se modifica. El único archivo que se toca es `Tienda.java`.

Parte 1: ver el problema

Ejecutar `Tienda` tal como viene y mirar `Tienda.java`. Razonar las siguientes preguntas.

1. `mostrarCatalogo` y `valorTotal` tienen tres bucles cada uno. ¿Qué cambia entre un bucle y el siguiente, además del tipo de la variable?
2. `mostrarDetalle()` y `getPrecioFinal()` se llaman en los tres bucles. ¿En qué clase están declarados los dos?
3. La tienda quiere sumar indumentaria al catálogo ¿Cuántos lugares de `Tienda.java` habría que tocar para que aparezca en el listado, en el conteo y en el valor total?

Parte 2: aplicar polimorfismo

Los pasos están marcados como TODO dentro de `Tienda.java`. En orden.

- TODO 1 (`mostrarCatalogo`): reemplazar los tres parámetros por un único `ArrayList<Producto> catalogo` y los tres bucles por uno solo. Después hay que cambiar la declaración del catálogo en `main` por una sola lista `ArrayList<Producto>` con los cinco productos mezclados.
- TODO 2 (`contarProductos`): con una sola lista, el método es una línea.
- TODO 3 (`valorTotal`): un solo bucle que acumule `getPrecioFinal()`.



Ejecutar y comprobar que la salida es la misma que antes. En `producto.mostrarDetalle()`, la variable es de tipo `Producto`, que es abstracta y no tiene esa clase ningún objeto. ¿Quién decide qué versión del método se ejecuta, y en qué momento?

Parte 3: probar

- TODO 4: escribir `masCaro(ArrayList<Producto> catalogo)`, que recorre la lista y devuelve el `Producto` de mayor precio final. Devuelve un `Producto`, no un `Libro`, quien lo llama no necesita saber cuál es. Llamarlo desde `main` y pedirle el detalle al que devuelve.
- TODO 5: crear `Indumentaria.java`. Hereda de `Producto`, tributa el 21%, y agrega `talle` y `color`. Implementar `getPrecioFinal()` y sobrescribir `mostrarDetalle()` reutilizando `super.mostrarDetalle()`.
- TODO 6: agregar una prenda a la lista `catalogo` de `main`. Una sola línea.

Ejecutar. La prenda tiene que aparecer en el listado, en el conteo, en el valor total y, si es la más cara, en `masCaro`. Antes de seguir, contar cuántos de los métodos escritos en la parte 2 hubo que modificar para que eso pasara, y comparar con la respuesta que se dio en la pregunta 3 de la parte 1.